ENTREGA GRUPAL / DISEÑO E IMPLEMENTACIÓN

SmartParking

Parte 1 · Diseño arquitectónico y decisiones

Equipo de RAT 2 · 28 de octubre de 2026

Solución educativa para administrar clientes, vehículos y reservas. Se implementa un monolito modular con autenticación propia, almacenamiento de identidad independiente y entrega durable de eventos al log.

Alcance y reglas acordadas

Entidad	Relación y regla principal
Cliente	Una única entidad permite actuar como propietario y como reservante. Cada registro se vincula a una identidad mediante identityUserId.
Vehículo	Pertenece obligatoriamente a un cliente. Matrícula única. El propietario permanece fijo en esta entrega.
Reserva	Vincula un vehículo con el cliente que lo reserva, distinto de su propietario. inicio < fin; estados ACTIVA o CANCELADA.

> **Aclaración del dominio**
> La regla de pertenencia del PDF original se interpreta con la aclaración del docente transmitida al equipo: un cliente reserva el auto de otro cliente. Una persona puede cumplir ambos papeles. Los intervalos son [inicio, fin): dos reservas contiguas son válidas.

Elección tecnológica

Herramienta	Justificación breve
TypeScript / Node.js / Express	Continuidad con el estilo de clase; módulos pequeños, contratos explícitos y validación con Zod.
PostgreSQL + pg	Transacciones y restricción de exclusión para coordinar reservas entre procesos. SQL parametrizado sin un ORM adicional.
PM2 / SQLite / k6	PM2 replica API y administra procesos; SQLite conserva el log independiente; k6 verifica el escenario de consultas.

Lectura del informe

A1 componentes y contratos: páginas 2-4. **A2** escenarios de calidad: página 5. **A3** decisión sobre comunicación de eventos: página 6. **A4** evidencia y revisión de consistencia: páginas 7-8.

Repositorio y versión: github.com/odllakoff/smartparking · d86fe22. El documento describe esa implementación; las pruebas y sus condiciones están versionadas.

A1 / COMPONENTES

Vista UML de componentes y persistencias

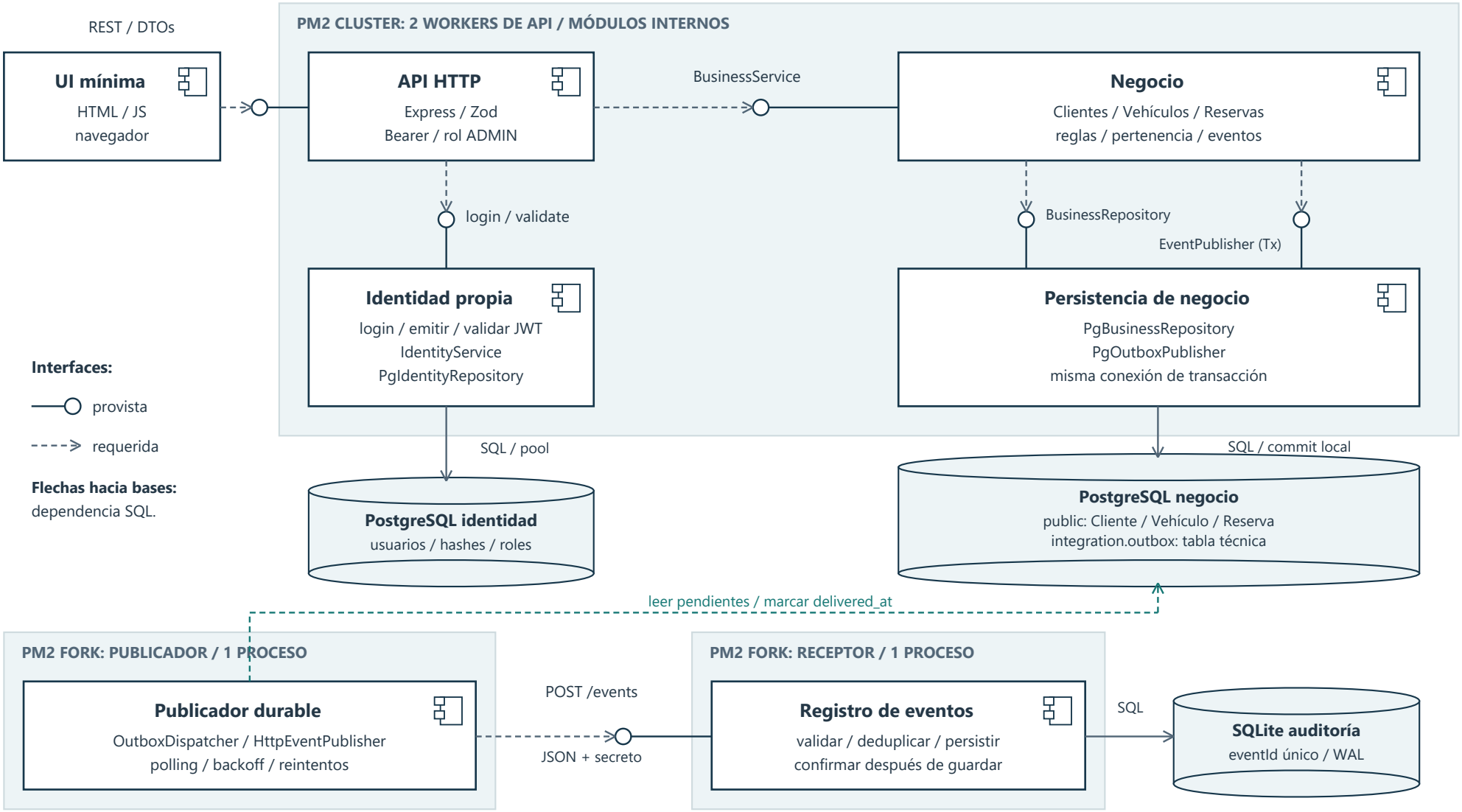


Figura 1. Vista agrupada de la implementación. Los círculos representan interfaces provistas; las dependencias discontinuas indican interfaces requeridas. Identidad agrupa su servicio y adaptador SQL; Persistencia de negocio agrupa los dos adaptadores que comparten transacción. El log final conserva su almacenamiento independiente.

A1 / INTERFACES

Contratos explícitos y datos de intercambio

Las interfaces REST y de casos de uso son contratos arquitectónicos. BusinessRepository, IdentityRepository y EventPublisher son también interfaces TypeScript. No se requiere una interfaz por cada clase.

Contrato provisto	Consumidor y operaciones
REST de negocio / API HTTP	UI o cliente HTTP. CRUD de clientes y vehículos; crear, modificar, consultar y cancelar reservas. JSON con Bearer token. Listas paginadas (20 por defecto, máximo 100).
IdentityService	API HTTP: login(email, password), validate(token). La validación devuelve userId y role; login devuelve AuthResponse.
IdentityRepository	Identidad: findByEmail y create; Negocio: exists al vincular un Cliente. PgIdentityRepository accede solo al storage de identidad.
BusinessRepository	BusinessService: búsquedas y CRUD. transaction(work) proporciona el repositorio y un EventPublisher sobre la misma conexión.
EventPublisher.publish(event)	Dentro del CRUD, PgOutboxPublisher persiste el pendiente; dentro del publicador, HttpEventPublisher envía el mismo DTO al receptor.
POST /events	HttpEventPublisher requiere JSON y secreto de servicio. Registro responde 204 después de persistir; 400 por contrato inválido y 401 por secreto incorrecto.

DTOs mínimos de respuesta y comunicación

DTO	Campos y tipos
ClienteDTO	id: UUID; nombre, email, telefono: string; identityUserId: string (UUID en la implementación).
VehiculoDTO	id, clienteId: UUID; matricula, marca, modelo: string.
ReservaDTO	id, clienteId, vehiculoId: UUID; inicio, fin: datetime; estado: ACTIVA | CANCELADA.
LoginRequest / AuthResponse	email, password: string / accessToken: JWT; expiresIn: number (segundos).
EventoAuditoriaDTO	eventId, entidadId: UUID; tipo, entidad, userId: string; fechaHora: datetime. userId se implementa como UUID.

Entradas y errores

Las altas omiten IDs generados. Las ediciones aceptan campos editables y mantienen fijos propietario, identidad vinculada, reservante y vehículo de la reserva. Fechas ISO 8601 con zona horaria, normalizadas a UTC. Estados HTTP: 400 validación, 401 identidad, 403 permisos, 404 inexistente, 409 conflicto y 503 dependencia temporalmente indisponible.

A1 / SEGURIDAD Y FLUJOS

Identidad, permisos y operaciones exitosas

Autenticación diseñada por el equipo

1. La API recibe credenciales y IdentityService busca el usuario en su repositorio propio. **2.** Compara la contraseña mediante scrypt con sal; Cliente no almacena credenciales. **3.** Emite JWT HS256 con sub=userId, role, issuer, audience y vencimiento de una hora.

4. En cada solicitud, el middleware exige Bearer y verifica firma, algoritmo permitido, emisor, audiencia y vigencia. **5.** Para operaciones propias, Negocio resuelve Cliente por identityUserId=sub y compara la pertenencia. **6.** La consulta global exige ADMIN antes de acceder al repositorio.

Requisito / recurso	CLIENTE	ADMIN
RF1 · Clientes	Sin acceso al CRUD.	CRUD de clientes.
RF2 · Vehículos	Gestiona sus vehículos. Catálogo ajeno mínimo para reservar.	Gestiona todos.
RF3 · Reservas	Crea sobre vehículo ajeno; modifica y cancela sus reservas.	Gestiona reservas de cualquier cliente.
RF4 · Consulta propia	Cliente derivado del JWT. clienteId en query se rechaza.	Usa la consulta global; no tiene Cliente implícito.
RF5 · Consulta global	Acceso rechazado.	Filtros por cliente, estado y fechas.

Crear una reserva y conservar su evento

1. Validar entrada e identidad. Obtener el reservante autenticado, salvo gestión explícita de ADMIN. **2.** Abrir transacción, verificar cliente y vehículo; rechazar auto propio. **3.** Guardar la reserva: PostgreSQL aplica CHECK de fechas y EXCLUDE GiST sobre vehículo e intervalo activo. Una carrera o edición solapada recibe 409.

4. Guardar EventoAuditoriaDTO en integration.outbox mediante la misma conexión. Si falla, rollback de ambos cambios. **5.** Confirmar commit y responder a la API. **6.** El publicador entrega después al log y marca delivered_at tras el acuse. El receptor deduplica por eventId.

> **Semántica de las operaciones**
> La respuesta exitosa confirma negocio y evento pendiente. El log se actualiza de forma eventual. Cancelar conserva la reserva y libera su intervalo; repetir una cancelación no produce otro evento. Las bajas con referencias se rechazan sin cascadas.

El propietario del vehículo no puede modificar reservas de terceros. Las altas de identidad son administrativas, sin registro público. El catálogo omite datos personales del propietario. Las claves de firma y del receptor se configuran fuera del código.

A2 / CALIDAD

Dos escenarios verificables

Escenario 1 · Performance de consultas

Elemento	Definición
Fuente / estímulo	Clientes y administradores autenticados generan 500 consultas de reservas por segundo.
Artefacto / ambiente	API, repositorio e índices; operación normal con dependencias disponibles. Dataset de 20 clientes, 100 vehículos y 10.000 reservas; generación y servicios en el equipo local documentado.
Respuesta	Resolver consultas propias y globales filtradas, autorizadas y paginadas, sin errores inesperados.
Medida	p95 < 300 ms durante 60 s estables; además, cero iteraciones descartadas, cero fallos HTTP y todos los checks correctos.
Táctica SAP / ubicación	Introducir concurrencia: dos workers PM2. Administrar recursos: pools y límite de página. Reducir demanda: índices, consultas acotadas y clave JWT reutilizada. Ubicación: ecosystem.config.cjs, db.ts, sql/business.sql, repository.ts e identity.ts.
Verificación	k6 constant-arrival-rate. Ensayo vigente: p95 25,67 ms, cero errores y cero descartes; ver página 7.

Escenario 2 · Integrabilidad y modificabilidad

Elemento	Definición
Fuente / estímulo	Docente o equipo solicita sustituir el transporte HTTP de auditoría por otro adaptador o broker, manteniendo el contrato de evento.
Artefacto / ambiente	Frontera EventPublisher y composición audit/worker.ts, durante mantenimiento de esta versión.
Respuesta	Agregar el adaptador y sustituir su composición; preservar CRUD, autorización, persistencia de negocio y contratos REST.
Medida	Cero cambios en service.ts, repository.ts y rutas HTTP; misma suite funcional aprobada y EventoAuditoriaDTO aceptado por una prueba de contrato del nuevo adaptador.
Táctica SAP / ubicación	Encapsular y restringir dependencias: servicios aislados del transporte. Interfaces explícitas y adaptador intermediario: EventPublisher, domain.ts, audit/publisher.ts y audit/worker.ts.
Verificación prevista	Ejecutar la prueba de contrato para cada adaptador incorporado y la integración general. El reemplazo por un broker es un escenario de cambio, no una funcionalidad ya implementada.

Las medidas distinguen lo observado de lo propuesto. La carga de consultas ya se ejecutó; la sustitución por un broker queda preparada como cambio verificable.

A3 / DECISIÓN ARQUITECTÓNICA

Comunicación durable de eventos

ADR-02 · Aceptada e implementada. Reemplaza ADR-01, que conserva la decisión inicial de HTTP posterior al commit. Se mantiene la numeración del repositorio para preservar la trazabilidad del entregable A3.

Problema y alternativas

Una operación puede confirmar su cambio y caer antes de notificar al log. Los reintentos en memoria no sobreviven al reinicio. La solución debe conservar la notificación de cada cambio confirmado y permitir cambiar su transporte.

Alternativa	Ventaja	Consecuencia principal
HTTP después del commit	Pocas piezas y confirmación directa.	Ventana de pérdida entre commit y publicación; el CRUD espera al receptor.
RabbitMQ directo	Mensajes aceptados por el broker pueden persistir y esperar consumidor.	Persisten dos escrituras sin coordinación atómica entre PostgreSQL y broker; agrega operación de colas.
Outbox + polling + HTTP	Cambio y pendiente comparten commit; recuperación de entregas tras reiniciar.	Tabla técnica adicional, publicador y entrega eventual; requiere deduplicación.

Decisión y mecanismo

Adoptar outbox transaccional. BusinessRepository.transaction entrega repositorio y EventPublisher sobre la misma conexión. PgOutboxPublisher guarda el evento. Un proceso PM2 fork toma un pendiente con FOR UPDATE SKIP LOCKED, envía por HTTP con timeout de 1.500 ms y marca delivered_at después de recibir 204.

Si falla la entrega, conserva eventId y registra attempts, last_error y next_attempt_at. Reintenta sin límite, con espera exponencial de 1 a 60 s y polling de 500 ms. Si cae tras la persistencia del receptor, el siguiente intento se deduplica por eventId. El log utiliza SQLite con WAL y synchronous=FULL.

> **Decisión explícita sobre persistencia**
> Se conservan tres entidades del dominio en public y se agrega una cuarta tabla física técnica: integration.outbox, en la misma base para compartir el commit. Esta es una interpretación documentada del equipo; no se presenta como una aprobación adicional del docente. El log final sigue independiente.

Atributos afectados y límites

Confiabilidad y disponibilidad: el pendiente sobrevive al reinicio y el CRUD puede operar con el receptor caído. **Performance:** un INSERT adicional en escrituras; consultas sin publicación. **Modificabilidad e integrabilidad:** transporte detrás de EventPublisher y DTO estable.

Entrega al menos una vez, sin orden estricto. Presupone conservar volúmenes, ejecutar el publicador y reparar fallos permanentes. El publicador mantiene un lock durante cada intento acotado, con pool propio de dos conexiones. Se conservan filas entregadas para inspección; no hay retención automática.

A4 / EVIDENCIA

Rendimiento y recuperación verificados

Resultado vigente con outbox activo
500 consultas/s durante 60 s. p95 = **25,67 ms**. Cero errores HTTP, cero iteraciones descartadas y todos los checks correctos.

Condiciones de la medición

Variable	Condición
Equipo / ejecución	AMD Ryzen 5 Microsoft Surface Edition, 6 núcleos / 12 hilos, 16,6 GB RAM decimal; Windows y Docker Desktop. Generador y servicios comparten equipo.
Versiones / procesos	Node v24.15.0; k6 v2.2.0; PostgreSQL 17-alpine. Dos workers API, un publicador y un receptor. Pools API: máximo 10 por base y worker; publicador: dos conexiones.
Datos / mezcla	20 clientes, 100 vehículos, 10.000 reservas. Aproximadamente 75% consultas propias y 25% globales, variando cliente, estado y offset; límite de 20 filas e intersección temporal.
Protocolo k6	21 logins en setup, fuera de la medición. Calentamiento 100/s por 10 s, pausa 2 s y 500/s por 60 s. 300 VUs preasignados, máximo 300. Se filtran métricas phase:steady.

Ensayos conservados

Configuración	p95	Descartes	Resultado
1 worker; clave JWT string	1.179,39 ms	6.627	No cumple
1 worker; clave reutilizada	764,45 ms	1.044	No cumple
2 workers; 100 VUs iniciales	155,50 ms	114	No cumple
2 workers; 300 VUs iniciales	242,75 ms	0	Cumple
2 workers + outbox; 300 VUs	25,67 ms	0	Cumple

El equipo compartido varía entre ejecuciones: la diferencia de p95 no demuestra que outbox acelere lecturas. El último escenario completó 30.001 consultas en steady por el límite de temporización. El promedio global de k6 incluye setup y calentamiento; no representa la tasa estable. No se midió capacidad máxima de escrituras o de vaciado de backlog.

Pruebas y fallos

5 pruebas unitarias; **9** pruebas/subpruebas de integración general; **7** de durabilidad. Carrera de 12 altas del mismo intervalo: una aceptada y once conflictos, observando ambos workers. La suite durable prueba rollback, caída tras commit, receptor caído, reinicio y caída tras acuse con deduplicación.

En una repetición el proceso de pruebas devolvió un fallo genérico sin informar casos; no se identificó la causa. La ejecución TAP y la repetición posterior del comando habitual aprobaron todos los casos. Comprobación final de demo: 12 eventos persistidos, 12 entregados y cero pendientes.

Evidencia: results/public/20261028-200851-2-workers.json y metadatos del mismo prefijo. Las copias públicas conservan métricas y checks y excluyen datos de setup con tokens.

A4 / CONSISTENCIA

Trazabilidad y cierre del diseño

Requisito	Aplicación en el diseño	Evidencia
RF1	BusinessService: clientes; acceso ADMIN.	CRUD temporal y rechazo CLIENTE.
RF2	Vehículos; propietario desde identidad o gestión ADMIN.	CRUD propio, rechazo ajeno y baja referenciada.
RF3	Reservas; fechas, propiedad y exclusión PostgreSQL.	Alta, edición, cancelación, contiguos y carrera.
RF4	Cliente.identityUserId = JWT.sub.	Consulta propia y rechazo clienteId suministrado.
RF5	requireAdmin + filtros por cliente, estado y fechas.	Rechazo CLIENTE y filtros globales.
Identidad propia	IdentityService y repositorio separado del negocio.	Login, firma, vigencia y audiencia.
Log independiente	Receptor con secreto y storage SQLite.	Persistencia y eventId único.
Entrega durable	Commit de negocio + outbox; publicador recuperable.	Rollback, muerte de procesos y reenvío idempotente.
Performance	PM2, pools, índices y consulta acotada.	k6: 500/s y p95 25,67 ms.
Cambio de transporte	EventPublisher y composición en worker.ts.	Frontera explícita; prueba de contrato prevista al agregar adaptador.

Coherencia y alcance

El UML muestra los mismos límites usados por las tácticas y por el ADR: identidad separada, negocio transaccional, publicador independiente y log idempotente. El evento pendiente comparte almacenamiento y commit con el negocio; el registro final permanece separado. ADMIN y pertenencia se verifican en servidor.

La solución mantiene una UI mínima y comentarios en español sobre reglas y decisiones. Esta entrega no incluye transferencia de propietario, cambio de reservante, renovación/revocación de tokens, borrado de reservas ni un broker. La instalación desde un clon limpio y el recorrido manual completo de formularios quedan como verificaciones de entrega pendientes; las suites funcionales, de durabilidad y de carga descritas sí se ejecutaron.

Fuentes y material reproducible

1. Consigna local: Prueba_Arquitectura_SmartParking_Parte1.pdf y aclaración del docente transmitida por el equipo.
2. Bass, L.; Clements, P.; Kazman, R. *Software Architecture in Practice*, 4.ª ed.: escenarios de calidad, tácticas e interfaces.
3. PostgreSQL 17: rangos y exclusión; SELECT y SKIP LOCKED.
4. Transactional outbox y Polling publisher (Chris Richardson).
5. PM2: cluster mode; k6: constant-arrival-rate.

Repositorio: github.com/odllakoff/smartparking · d86fe22. Fuentes editables: docs/ARQUITECTURA.md, docs/ADR-02.md, docs/componentes.puml y scripts/build-report.py. Pruebas: tests/ y load/queries.js. Instrucciones: README.md.